



Library Management System Project Report

Title Page

Project Title: Library Management System

Type: REST API-Based Web Application

Technology Stack: Spring Boot, MySQL, Spring Data JPA, Spring Web, ReactJS, REST APIs

Submitted By: Your Name

Course: B.E. CSE

Academic Year: 2025–2026

1. Abstract

The Library Management System is a REST API-based application designed to manage library-related data in a structured and efficient way. It supports operations such as adding, updating, deleting, and viewing authors, books, and user profiles through clean layered architecture. The project is built using Spring Boot for backend services, MySQL for data storage, and ReactJS for frontend interaction. It also demonstrates proper handling of different input field types and API testing using Postman.

2. Introduction

2.1 Project Overview

The main purpose of this project is to automate common library operations and provide a simple digital system for managing books, authors, and user profiles. Instead of using manual records, the application stores and retrieves data through REST APIs and a relational database. The system is designed with separate layers for controller, service, repository, and entity to keep the code maintainable and easy to understand.

2.2 Problem Statement

Manual library management is time-consuming and prone to errors. Searching for records, updating details, and managing relationships between books and authors becomes difficult when data is stored in physical registers or disconnected systems. This project solves that problem by providing a centralized REST-based application with efficient CRUD operations and database support.

2.3 Objectives

The main objectives of the project are:

- To build a REST API-based library management application.
- To perform CRUD operations for authors, books, and user profiles.
- To implement one-to-many relationship between authors and books.
- To test APIs using Postman.
- To handle different input types in request payloads.
- To maintain a clean layered architecture for better code organization.

3. Scope of the Project

This project covers the core library management features required for academic submission and portfolio use. It includes:

- Author management.
- Book management.
- User profile management.
- Search and filter support.
- REST API integration.
- Database persistence using MySQL and JPA.

The project is suitable for a college-level full-stack demonstration because it shows backend design, frontend interaction, and database handling in one application.

4. System Architecture

The application follows a standard layered architecture:

- **Client Layer:** ReactJS frontend or Postman.
- **Controller Layer:** Accepts HTTP requests.
- **Service Layer:** Contains business logic.
- **Repository Layer:** Communicates with the database.
- **Database Layer:** MySQL stores persistent data.

This separation improves readability, testing, and maintainability. The REST API flow is:
Client → Controller → Service → Repository → Database → JSON Response

5. Modules of the System

5.1 Author Management

This module handles adding, updating, deleting, and viewing author records. Each author can be linked with multiple books.

5.2 Book Management

This module manages books along with details like title, genre, ISBN, price, language, availability, and page count. Each book is associated with one author.

5.3 User Profile Management

This module demonstrates handling of different input field types such as text, email, password, radio buttons, checkboxes, dropdowns, date, datetime, number, and optional file name upload.

5.4 Search and Filter

This module allows users to search records based on text-based fields such as title or author name.

5.5 API Testing

Postman is used to verify all CRUD endpoints and confirm proper JSON request-response handling.

6. Technologies Used

Backend

- Spring Boot
- Spring Web
- Spring Data JPA
- Hibernate
- MySQL
- Validation API

Frontend

- ReactJS
- Axios
- HTML
- CSS

Tools

- Eclipse / Spring Tool Suite
- MySQL Workbench
- Postman
- VS Code

7. Database Design

7.1 Tables Used

- authors
- books
- user_profiles
- user_profile_interests

7.2 Relationships

- One author can have many books.
- One book belongs to one author.
- User interests are stored as a collection of values.

7.3 Key Design Points

- Primary keys are auto-generated.
- Foreign key mapping is used for author-book relationship.
- Validation is applied on request DTOs to prevent invalid input.

8. Functional Requirements

The system should:

- Add new authors.
- Add new books with author mapping.
- Add and manage user profiles.
- View all records.
- Update existing records.
- Delete records.
- Validate input data.
- Return JSON responses from APIs.

9. Non-Functional Requirements

- **Usability:** Simple and clean interface.
- **Maintainability:** Layered architecture keeps code organized.
- **Scalability:** REST APIs allow future expansion.
- **Performance:** Database operations are efficient for small to medium datasets.
- **Reliability:** Validation and exception handling improve stability.

10. Implementation Details

10.1 Backend Implementation

The backend is built using Spring Boot. Controllers receive HTTP requests and pass them to service classes. Services handle business logic and use repositories to interact with MySQL. Entities represent database tables, and DTOs are used for request validation.

10.2 Frontend Implementation

The frontend is built using ReactJS. Separate pages are created for authors, books, and users. Axios is used to connect the frontend with backend APIs. Forms are built using controlled components so data can be submitted correctly to the server.

10.3 Input Field Handling

The user profile module demonstrates support for:

- Text input.
- Email input.
- Password input.
- Radio selection.
- Checkbox multiple selection.
- Dropdown selection.
- Date input.
- Date-time input.
- Number input.
- File upload name handling.

11. API Testing

Postman was used to test all REST endpoints.

Tested Operations

- POST for creating records.
- GET for fetching records.
- PUT for updating records.
- DELETE for removing records.

Sample API Endpoints

- /api/authors
- /api/books
- /api/users

Validation Checks

- Valid request body format.
- Correct date and datetime values.
- Proper author-book mapping.
- Required fields present.
- Error handling for invalid or missing data.

12. Testing Summary

Test Case	Action	Expected Result	Status
TC1	Add Author	Author saved successfully	Pass
TC2	Add Book	Book saved with valid author ID	Pass
TC3	Add User	User profile saved correctly	Pass
TC4	Update Record	Record updated successfully	Pass
TC5	Delete Record	Record removed successfully	Pass
TC6	Fetch All Records	JSON list returned	Pass

13. Challenges Faced

Some common challenges during development were:

- Handling LocalDate and LocalDateTime in JSON requests.
- Managing one-to-many mapping between authors and books.

- Ensuring CORS works between frontend and backend.
- Formatting checkbox values as a list in request payloads.
- Matching frontend field names with backend DTO fields.

These were resolved by using DTOs, proper JSON formatting, and clean API integration.

14. Future Enhancements

The project can be improved further by adding:

- Login and authentication.
- Role-based access control.
- File upload for profile images or book covers.
- Issue and return book functionality.
- Fine calculation.
- Pagination and sorting.
- Advanced search filters.
- Dashboard analytics.

15. Conclusion

The Library Management System successfully demonstrates a clean REST API-based full-stack application with proper backend layering and database integration. It automates essential library operations and supports structured data handling through Spring Boot and MySQL. The project is useful for academic submission because it shows practical knowledge of API design, database relationships, validation, and frontend-backend integration.

16. Bibliography

- Spring Boot Documentation
- Spring Data JPA Documentation
- MySQL Documentation
- ReactJS Documentation
- Postman Documentation

17. Declaration

I hereby declare that this project report titled **Library Management System** is my original work completed as part of my academic project requirements. All sources used in the project have been properly acknowledged.

18. Acknowledgement

I would like to express my sincere gratitude to my faculty guide, department, and institution for their support and guidance during the completion of this project. I also thank everyone who helped me in completing this project successfully.

If you want, I can next turn this into:

- a VTU-style formal project report, or
- a shorter submission-ready version with title page, index, and viva points.



1. <https://www.scribd.com/document/975164386/Report-Template>
2. <https://www.scribd.com/document/762359826/Internship-report-Full-stack-Development>
3. <https://www.studocu.com/in/document/suresh-gyan-vihar-university/computer-science/kartikeya-full-stack-project-report/37668628>
4. [https://srmus.ac.in/view/governance/naac/1.3.4/Supporting Documents/MCA/Mohit Pradhan \(20IT103018\).pdf](https://srmus.ac.in/view/governance/naac/1.3.4/Supporting Documents/MCA/Mohit Pradhan (20IT103018).pdf)
5. <https://www.scribd.com/document/578153108/2018BCEX1058-TARUN-VYAS-PROJECT-REPORT>
6. <https://www.slideshare.net/slideshow/library-management-system-project-report/81022120>
7. <https://sjcit.ac.in/wp-content/uploads/2022/11/1SJ18CS085-Sameer-Pasha-R.pdf>
8. <https://www.scribd.com/document/690467783/SE-Project-Library-Management-System>
9. <https://www.scribd.com/document/735400266/Summer-Training-Report>
10. <https://www.scribd.com/document/611345786/library-management-system-final-report>